

Process Synchronization

Part 3

Sistemas de Computadores

Departamento de Engenharia Informática

Instituto Superior de Engenharia do Porto

Luís Nogueira (lmn@isep.ipp.pt)

Overview

- Starvation
- Readers-Writers problem

Introduction

- **In previous lectures, we introduced the need for process synchronization**
 - Concurrent processes access shared resources
 - Incorrect coordination leads to inconsistent system behavior
- **We studied the critical section problem and race conditions**
 - Multiple processes accessing shared data without proper control
 - Solution: enforce mutual exclusion (e.g., mutexes, semaphores)
- **We explored deadlocks and livelocks**
 - Deadlock: processes blocked forever (no progress)
 - Livelock: processes keep reacting but make no progress

Introduction

- **This lecture we discuss how to coordinate processes with different roles**
 - Readers (safe to run concurrently)
 - Writers (require exclusive access)

- **While addressing a new challenge**
 - Mitigating **starvation** in multi-role systems

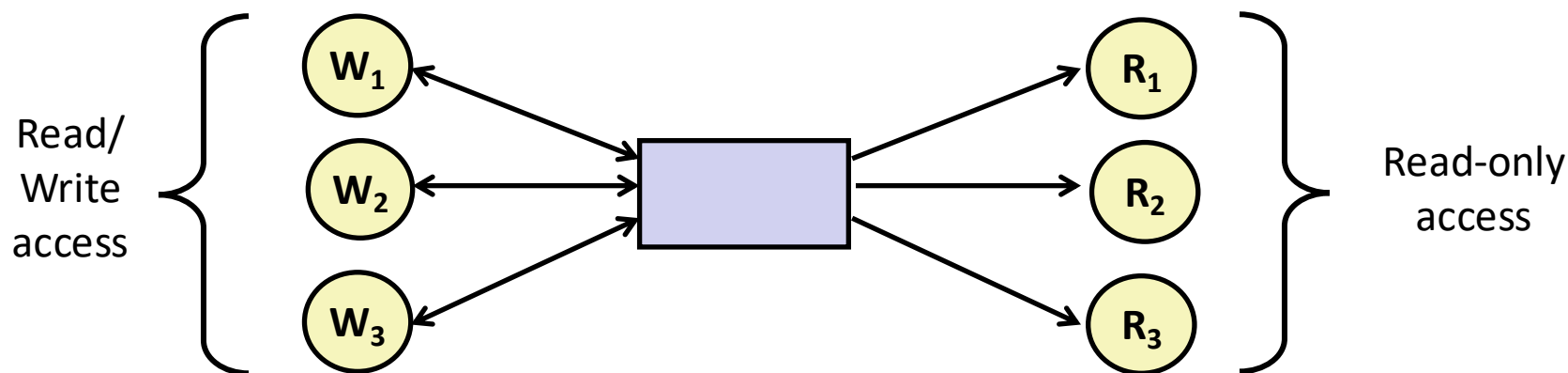
Starvation

- Occurs when processes are **indefinitely delayed** because others monopolize shared resources
- Leads to reduced throughput and can cause parts of the system to appear unresponsive
- **Mitigation strategies**
 - Algorithms designed to ensure every process eventually runs
 - Dynamically increasing a process's priority over time to force scheduling (aka priority aging, managed by the OS scheduler, discussed in a future lecture)

Readers-Writers problem

■ Problem statement:

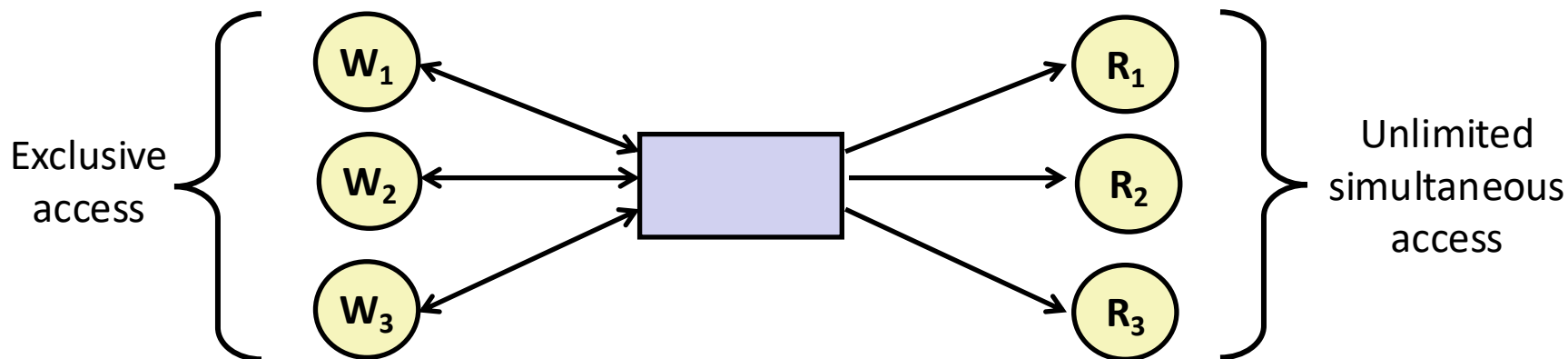
- Reader processes only read the shared data
- Writer processes modify the shared data (read/write access)



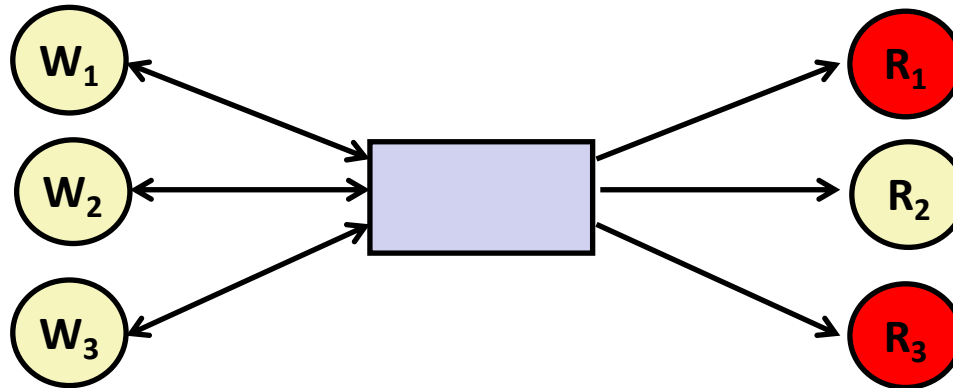
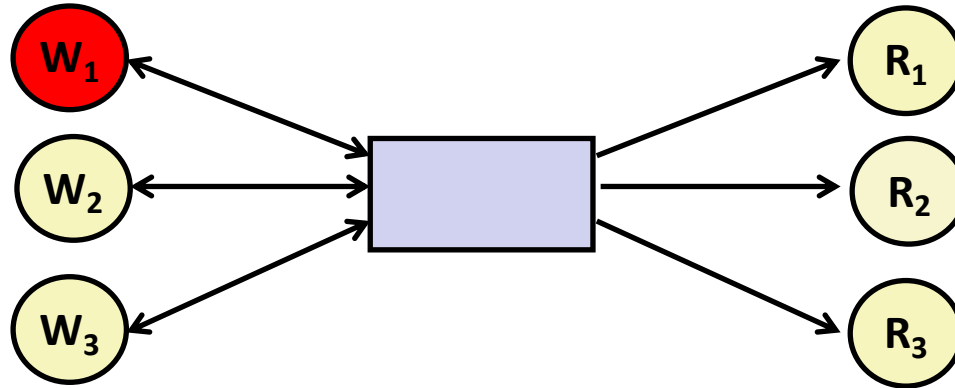
- Since it was originally stated, it has been used to test nearly every new synchronization primitive

Readers-Writers problem constraints

- Writers must have **exclusive access** to the shared data
- Unlimited number of readers can access the shared data **simultaneously**



Readers-Writers examples



Why does this matter in practice?

- **The readers–writers problem models real system design decisions**
 - Choosing the right strategy directly impacts performance and responsiveness
- **E.g., database systems**
 - Multiple clients execute read queries concurrently
 - Write operations (INSERT, UPDATE, DELETE) require exclusive access
 - Balance high read throughput with correct and consistent writes
- **Also appears in:**
 - File systems → many reads, controlled writes
 - OS kernels → shared data structures
 - Web services → frequent reads, rare updates

Variants of Readers-Writers problem

■ First readers-writers problem (favors readers)

- No reader should be kept waiting unless a writer has already been granted permission to use the object
- Readers are **not blocked by waiting writers**, which effectively gives readers priority and may delay writers indefinitely

■ Second readers-writers problem (favors writers)

- Once a writer is ready to write, it performs its write as soon as possible
- A **reader** that arrives after a writer **must wait**, even if the writer is also waiting

■ Notes:

- Starvation (where a process waits indefinitely) is possible in both cases
- Study the code examples provided on Moodle

Solution to first Readers-Writers problem

- Data in a region of memory shared by writer and reader processes

```
typedef struct{
    int readcnt;          /* number of readers in the critical section */
    int data;             /* data to be shared */
}shared_data;

/* exclusive access to readcnt */
mutex_rc = sem_open("/mutex_rc",O_CREAT,0666,1);

/* exclusive access to data write */
mutex_data = sem_open("/mutex_data",O_CREAT,0666,1);

/* assume s points to shared memory */
```

rw1.c

Solution to first Readers-Writers problem

```

void reader(){
    while(1){
        sem_wait(mutex_rc);
        s->readcnt++;
        if(s->readcnt == 1)                /* First in */
            sem_wait(mutex_data);
        sem_post(mutex_rc);

        /* Reading happens here */
        /* any number of readers */

        sem_wait(mutex_rc);
        s->readcnt--;
        if(s->readcnt == 0)                /* Last out */
            sem_post(mutex_data);
        sem_post(mutex_rc);
    }
}

```

rw1.c

```

void writer(){
    while(1){
        sem_wait(mutex_data);

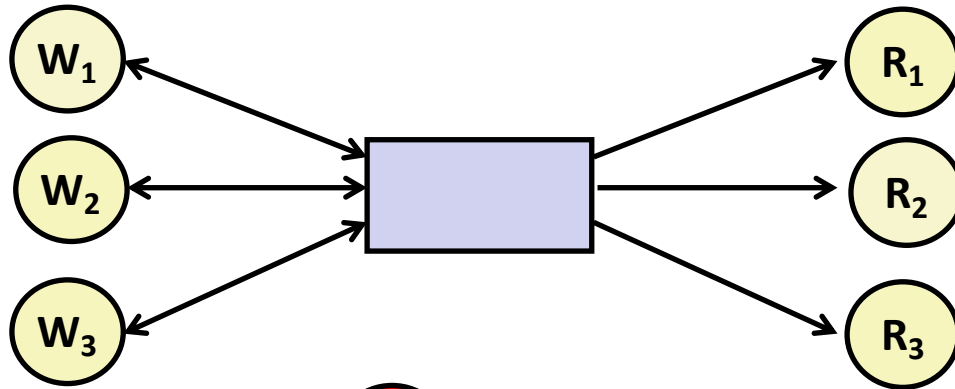
        /* Writing here */
        /* only one writer */

        sem_post(mutex_data);
    }
}

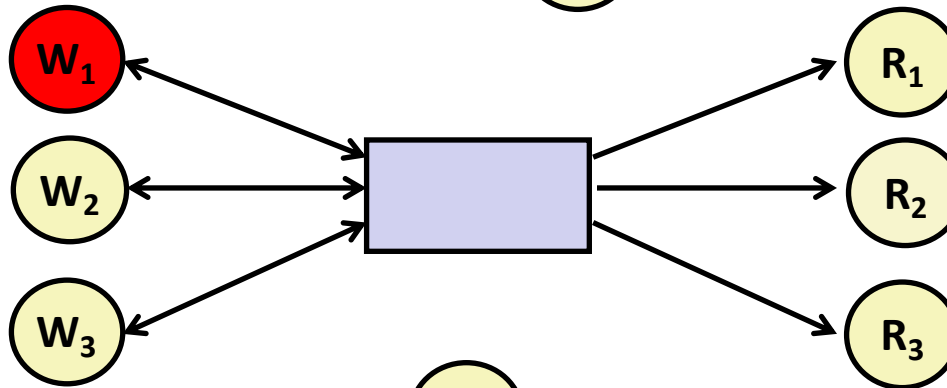
```

rw1.c

Readers-Writers examples

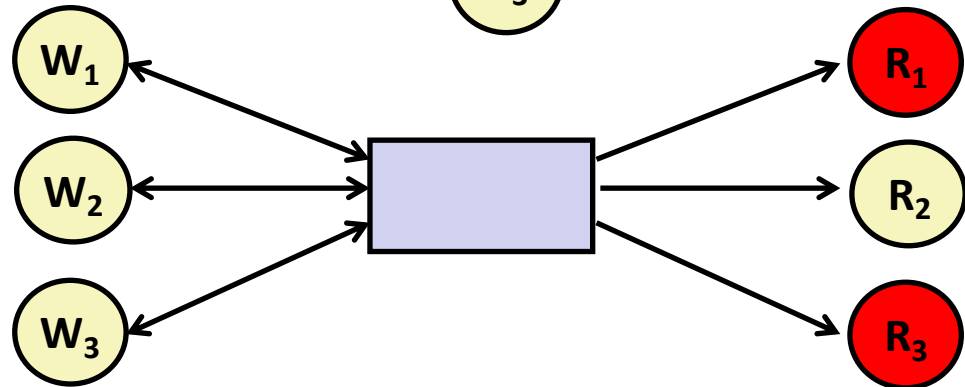


`mutex_data = 1`
`readcnt = 0`



`mutex_data = 0`
`readcnt = 0`

`mutex_data = 0`
`readcnt = 2`



Solution to first Readers-Writers problem

```

void reader(){
    while(1){
        sem_wait(mutex_rc);
        s->readcnt++;
        if(s->readcnt == 1)                /* First in */
            sem_wait(mutex_data);
        sem_post(mutex_rc);

        /* Reading happens here */
        /* any number of readers */

        sem_wait(mutex_rc);
        s->readcnt--;
        if(s->readcnt == 0)                /* Last out */
            sem_post(mutex_data);
        sem_post(mutex_rc);
    }
}

```

rw1.c

```

void writer(){
    while(1){
        sem_wait(mutex_data);

        /* Writing here */
        /* only one writer */

        sem_post(mutex_data);
    }
}

```

rw1.c

Arrivals: R1 R2 W1 R3

Solution to first Readers-Writers problem

```

void reader(){
    while(1){
        sem_wait(mutex_rc);
        s->readcnt++;
        if(s->readcnt == 1)                /* First in */
            sem_wait(mutex_data);
        sem_post(mutex_rc);

R1 → /* Reading happens here */
      /* any number of readers */

        sem_wait(mutex_rc);
        s->readcnt--;
        if(s->readcnt == 0)                /* Last out */
            sem_post(mutex_data);
        sem_post(mutex_rc);
    }
}
rw1.c

```

```

void writer(){
    while(1){
        sem_wait(mutex_data);

        /* Writing here */
        /* only one writer */

        sem_post(mutex_data);
    }
}

```

rw1.c

Arrivals: **R1** R2 W1 R3

readcnt == **1**
 mutex_data == **0**

Solution to first Readers-Writers problem

```

void reader(){
    while(1){
        sem_wait(mutex_rc);
R2 → s->readcnt++;
        if(s->readcnt == 1)           /* First in */
            sem_wait(mutex_data);
        sem_post(mutex_rc);

R1 → /* Reading happens here */
    /* any number of readers */

        sem_wait(mutex_rc);
        s->readcnt--;
        if(s->readcnt == 0)           /* Last out */
            sem_post(mutex_data);
        sem_post(mutex_rc);
    }
}
rw1.c

```

```

void writer(){
    while(1){
        sem_wait(mutex_data);

        /* Writing here */
        /* only one writer */

        sem_post(mutex_data);
    }
}

```

rw1.c

Arrivals: R1 R2 W1 R3

readcnt == 2
mutex_data == 0

Solution to first Readers-Writers problem

```

void reader(){
    while(1){
        sem_wait(mutex_rc);
        s->readcnt++;
        if(s->readcnt == 1)                /* First in */
            sem_wait(mutex_data);
        sem_post(mutex_rc);

        /* Reading happens here */
        /* any number of readers */

        sem_wait(mutex_rc);
        s->readcnt--;
        if(s->readcnt == 0)                /* Last out */
            sem_post(mutex_data);
        sem_post(mutex_rc);
    }
}

```

rw1.c

```

void writer(){
    while(1){
        sem_wait(mutex_data); ← W1

        /* Writing here */
        /* only one writer */

        sem_post(mutex_data);
    }
}

```

rw1.c

Arrivals: R1 R2 **W1** R3

```

readcnt == 2
mutex_data == 0

```

Solution to first Readers-Writers problem

```

void reader(){
    while(1){
        sem_wait(mutex_rc);
        s->readcnt++;
        if(s->readcnt == 1)                /* First in */
            sem_wait(mutex_data);
        sem_post(mutex_rc);

        /* Reading happens here */
        /* any number of readers */

        sem_wait(mutex_rc);
        s->readcnt--;
        if(s->readcnt == 0)                /* Last out */
            sem_post(mutex_data);
        sem_post(mutex_rc);
    }
}

```

rw1.c

```

void writer(){
    while(1){
        sem_wait(mutex_data); ← W1

        /* Writing here */
        /* only one writer */

        sem_post(mutex_data);
    }
}

```

rw1.c

Arrivals: R1 R2 W1 R3

readcnt == 1
mutex_data == 0

Solution to first Readers-Writers problem

```

void reader(){
    while(1){
        sem_wait(mutex_rc);
        R3 → s->readcnt++;
        if(s->readcnt == 1)                /* First in */
            sem_wait(mutex_data);
        sem_post(mutex_rc);

        R2 → /* Reading happens here */
              /* any number of readers */

        sem_wait(mutex_rc);
        s->readcnt--;
        if(s->readcnt == 0)                /* Last out */
            sem_post(mutex_data);
        sem_post(mutex_rc);
    }
}
rw1.c

```

```

void writer(){
    while(1){
        sem_wait(mutex_data); ← W1

        /* Writing here */
        /* only one writer */

        sem_post(mutex_data);
    }
}
rw1.c

```

Arrivals: R1 R2 W1 **R3**

readcnt == **2**
 mutex_data == 0

Solution to first Readers-Writers problem

```

void reader(){
    while(1){
        sem_wait(mutex_rc);
        s->readcnt++;
        if(s->readcnt == 1)                /* First in */
            sem_wait(mutex_data);
        sem_post(mutex_rc);

        /* Reading happens here */
        /* any number of readers */

        sem_wait(mutex_rc);
        s->readcnt--;
        if(s->readcnt == 0)                /* Last out */
            sem_post(mutex_data);
        sem_post(mutex_rc);
    }
}

```

rw1.c

```

void writer(){
    while(1){
        sem_wait(mutex_data); ← W1

        /* Writing here */
        /* only one writer */

        sem_post(mutex_data);
    }
}

```

rw1.c

Arrivals: R1 R2 W1 R3

readcnt == 1
mutex_data == 0

Solution to first Readers-Writers problem

```

void reader(){
    while(1){
        sem_wait(mutex_rc);
        s->readcnt++;
        if(s->readcnt == 1)                /* First in */
            sem_wait(mutex_data);
        sem_post(mutex_rc);

        /* Reading happens here */
        /* any number of readers */

        sem_wait(mutex_rc);
        s->readcnt--;
        if(s->readcnt == 0)                /* Last out */
            sem_post(mutex_data);
        sem_post(mutex_rc);
    }
}

```

rw1.c

```

void writer(){
    while(1){
        sem_wait(mutex_data); ← W1
        /* Writing here */
        /* only one writer */

        sem_post(mutex_data);
    }
}

```

rw1.c

Arrivals: R1 R2 W1 R3

readcnt == 0
mutex_data == 1

Solution to first Readers-Writers problem

```

void reader(){
    while(1){
        sem_wait(mutex_rc);
        s->readcnt++;
        if(s->readcnt == 1)                /* First in */
            sem_wait(mutex_data);
        sem_post(mutex_rc);

        /* Reading happens here */
        /* any number of readers */

        sem_wait(mutex_rc);
        s->readcnt--;
        if(s->readcnt == 0)                /* Last out */
            sem_post(mutex_data);
        sem_post(mutex_rc);
    }
}

```

rw1.c

```

void writer(){
    while(1){
        sem_wait(mutex_data);

        /* Writing here */
        /* only one writer */
        ← W1

        sem_post(mutex_data);
    }
}

```

rw1.c

Arrivals: R1 R2 W1 R3

readcnt == 0
mutex_data == 0

Correctness guarantees

■ Mutual exclusion for writers

- Only one writer can access the shared data at a time
- Enforced by the `mutex_data` semaphore

■ Concurrent access for readers

- Multiple readers can access the shared data simultaneously
- No interference between readers

■ Readers vs writers coordination

- First reader blocks writers (`sem_wait(mutex_data)`)
- Last reader releases writers (`sem_post(mutex_data)`)

Correctness guarantees

■ Protection of shared variables

- `readcnt` is safely updated using `mutex_rc`
- Prevents race conditions on the reader counter

■ At any time:

- Either multiple readers are inside the critical section
- Or one writer is inside
- **Never both**

■ Continuous stream of readers will block writers indefinitely (writers' starvation)

- This can lead to delayed or blocked writer processes, potentially impacting system responsiveness

Second readers-writers problem

- **The second readers-writers solution addresses the limitations of the first approach**
 - New writers block subsequent readers from entering the critical section
- **Trade-off**
 - A continuous stream of writers can lead to reader starvation
- **Preferred when write-priority and low latency for updates are critical**

Solution to second Readers-Writers problem

- Data in a region of memory shared by writer and reader processes

```
typedef struct{
    int readcnt;          /* number of readers in the critical section */
    int writecnt;         /* number of waiting writers */
    int data;             /* data to be shared */
}shared_data;

/* exclusive access to readcnt and writecnt */
mutex_rc = sem_open("/mutex_rc",O_CREAT,0666,1);
mutex_wc = sem_open("/mutex_wc",O_CREAT,0666,1);

/* indicates a reader is trying to enter */
try_read = sem_open("/tr",O_CREAT,0666,1);

/* exclusive access to data write */
mutex_data = sem_open("/swrite",O_CREAT,0666,1);
```

rw2.c

Solution to second Readers-Writers problem

```

void reader(){
    while(1){
        sem_wait(try_read);
        sem_wait(mutex_rc);
        s->readcnt++;
        if(s->readcnt == 1)
            sem_wait(mutex_data);
        sem_post(mutex_rc);
        sem_post(try_read);

        /* Reading happens here */
        /* any number of readers */

        sem_wait(mutex_rc);
        s->readcnt--;
        if(s->readcnt == 0)
            sem_post(mutex_data);
        sem_post(mutex_rc);
    }
}

```

rw2.c

```

void writer(){
    while(1){
        sem_wait(mutex_wc);
        s->writecnt++;
        if(s->writecnt == 1)
            sem_wait(try_read);
        sem_post(mutex_wc);

        sem_wait(mutex_data);
        /* Writing here */
        /* only one writer */
        sem_post(mutex_data);

        sem_wait(mutex_wc);
        s->writecnt--;
        if(s->writecnt == 0)
            sem_post(try_read);
        sem_post(mutex_wc);
    }
}

```

rw2.c

Solution to second Readers-Writers problem

■ Writers are given priority over readers

- Once a writer arrives, new readers are prevented from entering
- Key mechanism: `try_read` as a gate

■ No writers waiting

- Gate is open
- Readers pass freely and can run concurrently

■ When a writer arrives

- First writer closes the gate (`sem_wait(try_read)`)
- **New readers are blocked** at the gate
- Existing readers finish normally

Solution to second Readers-Writers problem

■ Writer execution

- Writers access the shared data one at a time (`sem_wait(mutex_data)`)
- Waiting writers proceed sequentially (`sem_post(mutex_data)`)

■ After the last writer

- Gate is reopened (`sem_post(try_read)`)
- Blocked readers are allowed to enter again

Starvation trade-offs

■ First solution (reader preference)

- Writers may starve due to continuous incoming readers

■ Second solution (writer preference)

- Readers may starve if writers keep arriving

■ Key takeaway

- Prioritization improves progress for one group at the cost of fairness for the other

Limitations of POSIX semaphores

■ No fairness guarantees

- The POSIX standard does not define the order in which waiting processes are resumed

■ Practical consequence

- You **cannot assume FIFO ordering**
- Starvation is still possible with naïve semaphore-based solutions

■ Design implication

- Fairness must often be implemented at user level
- Built on top of basic synchronization primitives provided by the OS

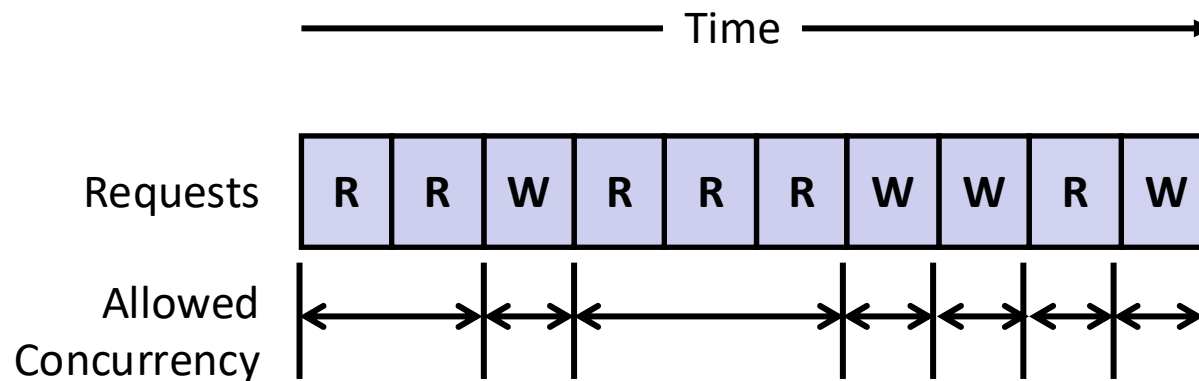
Readers–Writers with FIFO fairness

■ Goal: ensure fairness (no starvation)

- Requests are served in arrival order (FIFO)
- Guarantees bounded waiting time for all processes

■ Core Idea: ordered request queue

- All read/write requests are inserted into a FIFO queue
- A controller (implicit or explicit) grants access based on queue order



Readers–Writers with FIFO fairness

■ Read request

- Proceeds if no writer is active and all earlier requests are also reads
- Multiple readers may proceed together

■ Write request

- Proceeds only when it is at the front of the queue and no other process is accessing the resource

- **Refer to the example provided on Moodle for a hands-on demonstration of how these concepts are applied in practice**

Read-Write locks

- The readers–writers problem leads to a practical abstraction: read–write locks
- Widely supported in modern OSes and libraries
 - `rw_lock` (Linux kernel)
 - `pthread_rwlock_t` (POSIX)
 - `ReentrantReadWriteLock` (Java)
- A read–write lock must be acquired in one of two modes:
 - Read mode (**shared**)
 - Write mode (**exclusive**)

Read-Write locks

■ Read mode

- Multiple threads/processes may hold the lock simultaneously
- Suitable for operations that do not modify shared data

■ Write mode

- Only one thread/process may hold the lock
- Requires exclusive access to ensure correctness

■ At any time:

- Either multiple readers hold the lock
- Or one writer holds the lock
- **Never both**

When are they useful?

- **In applications where it is easy to identify:**
 - Which processes only read shared data
 - Which processes only write shared data

- **Increased concurrency in read-heavy workloads where writes are infrequent**
 - Multiple readers can proceed simultaneously
 - Improves throughput compared to mutexes

- **Trade-off: higher overhead**
 - More complex than simple mutexes
 - Additional coordination required to manage modes

POSIX Read-Write Lock

■ Operations:

- **Acquire read lock** – `pthread_rwlock_rdlock(pthread_rwlock_t *rwlock)`
 - **Acquire write lock** – `pthread_rwlock_wrlock(pthread_rwlock_t *rwlock)`
 - **Release (either) lock** – `pthread_rwlock_unlock(pthread_rwlock_t *rwlock)`
-
- **Up to the programmer to decide what requires read access and what requires write access**
 - Library must be used correctly!
-
- **Refer to the document “Example program: Linked list” available on Moodle**

Wrap up

■ Race conditions

- Property violated – Safety
- Symptom – Incorrect or inconsistent data
- Cause – Unsynchronized concurrent access
- Typical solution – Mutex / semaphore (mutual exclusion)
- Guarantee provided – At most one process in critical section

■ Deadlock

- Property violated – Progress (liveness)
- Symptom – Processes blocked forever
- Cause – Circular wait and resource holding
- Typical solution – Resource ordering, avoidance, detection and recovery
- Guarantee provided – System cannot reach circular wait

Wrap up

■ Livelock

- Property violated – Progress (liveness)
- Symptom – Processes remain active but make no useful progress
- Cause – Repeated reactions to each other's state changes (no blocking, but no progress)
- Typical solution – Backoff strategies, randomness, priority adjustment, retry delays
- Guarantee provided – System eventually makes progress (breaks repetitive interference loops)

Wrap up

■ Starvation

- Property violated – Fairness
- Symptom – A process waits indefinitely while others proceed
- Cause – Unfair scheduling or priority rules
- Typical solution – Priority adjustment, FIFO queues
- Guarantee provided – Bounded waiting time

Conclusion

- **Process synchronization is a fundamental concept in operating systems and concurrent programming**
 - It ensures that multiple processes can work together efficiently and without conflicts
 - By addressing issues like mutual exclusion, race conditions, deadlocks, and starvation, process synchronization plays a crucial role in maintaining system stability and data integrity
- **A solid understanding of process synchronization is essential for building robust and efficient concurrent systems**
 - Whether you're a systems programmer or a software developer

Conclusion

- Over the past two decades the research community has developed a body of knowledge concerning **lock-free** and **wait-free** algorithms and data structures
 - These techniques allow concurrent update of shared data structures without resorting to critical sections protected by OS managed locks
- Designing generalized lock-free and wait-free algorithms is hard (and outside the scope of this course)
 - Difficult to get right performance and correctness
- Well-established, tested, tuned implementations of common data structures are available for practical use